

How can we help you?

[Additional Info \(../../Content/Guide/r_additional_info.html?TocPath=Additional%20Info%7C____0\) >](#)
SDK User Guide

SDK USER GUIDE

1. INTRODUCTION

2. OVERVIEW

3. AUTHENTICATION

4. PROTOCOL DESCRIPTION

5. ENCRYPTION ALGORITHM FUNCTION DESCRIPTION

6. API DEFINITION

Get configuration of system

The configuration of the system includes site list, device list and worker list. Getting configuration will get the full configuration of system and data

dictionary. The configuration is the basic information which can help to filter historical data, subscribe real time data and understand relationships and identify patterns.

a. Get site configuration list

POST

/third/gas/v1/getSiteList Get full site configuration list of system

Request content-type= application/json {

"appid": "98bcd21-68c1-4216-84a4-d99d30cb61db", "data": { //only for definition, the content is in sign field

},

"random": 2457, //number, int4 "sign": "\$SIGN",

"uri": \$URI

}

Response content-type= application/text {

"rtData": {

"siteList": [{

"siteId": 76,

"name": "2020 Sales Kickoff", //site name "city":

"Houston", //city of site "timezone": "11", //

timezone of site "country": "United States", //

country of site

"countryName": "United States", //country name

"province": "TX", //province of site

"center":

"118.83461858749142,31.9560272454019",

//center longitude and latitude "zip": "77061", //

zip of site

"address1": "8181 Airport Blvd" //addr of site

}

],

“total”:10 //total num

}

}

//if response http status code is 400 or 403,
please modify parameters as requested

b. Get worker configuraton list

POST /third/gas/v1/ getWorkerList Get full
worker configuration list of system, or get a
worker configuration by filter with worker id

Request content-type= application/json {

"appid": "98bcd21-68c1-4216-84a4-
d99d30cb61db", "data": { //only for definition, the
content is in sign field

workerId:1 // option,worker id is from response of
this api. Null for all worker

}

},

"random": 2457, //number, int4 "sign": "\$SIGN",

"uri":\$URI

}

Response content-type= application/text {

"rtData": {

“workerList”:[{

"workerId": 198, //worker id

"userName": "Troy Walsh", //worker name

"firstName": "test_frist", //user first name

"middleName": "test_middle", //user middle name

"lastName": "test_last", //user last name

"email": "18951873142@sina.cn", //user email

"phone": "",

"sites": [{ //worker current location sites info

```

"id": 32, //site Id
"name": "Austin Fire Depart" // site name
}],
"status": 1, //1- active ;0-inactive
"nationCode": "1" // nation code , See appendix
}
],
"total":10 //total num
}
}

```

//if response http status code is 400 or 403,
please modify parameters as requested

c. Get devices configuraton list

POST /third/gas/v1/ getDeviceList Get all the
devices configuration list of system, or get some
devices configuration by filter with sensor Name
or serial No

```

Request content-type= application/json {
"appid": "98bcd21-68c1-4216-84a4-
d99d30cb61db", "data": { //only for definition, the
content is in sign field sensorName:" NOISE", //
option, sensor name of
device contains, See appendix serialNo:"
M03170111421" //option,device serialNo
"isOnline": 0, // option, online 1 ; offline 0; null for
all "siteld":23 //option
},
"random": 2457, //number, int4 "sign": "$SIGN",
"uri":$URI
}

```

Response content-type= application/text {

```
rtData": {  
    "deviceList": [ {  
        "id": 926, //device id  
        "isOnline": 0,  
        "onlineTime": "2020-04-13 07:54:58",  
        "offlineTime": "2020-04-13 08:15:43",  
        "nickName": "", // nick name of device  
        "assetType": "GasDetector", //device type  
        "assignSites": ["site"],  
        //device assigned sites  
        "serialNumber": "001F001231",  
        "status": 2, //1-in stock 2- active 3-in  
        //maintenance 4-archived  
        "name": "MultiRAE", //device name  
        "brand": "RAE",  
        //device brand  
        "model": "PGM-6220", // device  
        //model  
        "location": "Energy Point", //device location  
        "gps": {  
            "lat": 31.40383060508787,  
            "lng": 121.21138884741838  
        },  
        "assignedTo": [ "user2"  
    ],  
        "battery": 100, "alarmInfo": {  
        "alarms": [41], // just device level  
        "sensorAlarms": [  
            // just sensor level  
            {  
                "name": "CO", "alarm": 2  
            },  
            {  
                "name": "H2S", "alarm": 2  
            }  
        ]  
    }  
    }  
}
```

```

{
  "name": "SO2", "alarm": 2
},
{
  "name": "O2", "alarm": 2
}
]
},
"type": 1, // 0-fixed 1-portable 2-transportable
"sensorList": [
{
  "idx": 0,
  "alarm": [ 0
],
  "special": 0,
  "detectionMode": 0, // definition in 6.3 Detection
  Mode definition
  "val": 0.1,
  "decimalPoint": 1, // reading accuracy
  "name": "h2s", // sensor name "unit": "ppm" //
  sensor unit
}
]
},
],
"total": 10 // total num

```

7.2 Get real time data

7.2.1 Using websocket to setup connection

Uri is /third/ws/v1

The client needs to send ping message to server regularly, the time interval should be between 60-90 seconds.

7.2.2 Subscribe real time data

Real time data includes device data, worker data, alarm/alert event data and online/offline status data. Third-party system can subscribe the data required by subDataType in request. And Real time data can be subscribed by whole tenant or site or worker or device. It depends on the type fields in request body. One thing to notice is that this subscribe request must be send every 60 seconds to avoid information pushing waste caused by not unsubscribing in time.

Request (Must send this request every 60 seconds) {

"data": [// only for definition, content in sign

{

"type":1,// 1:tenant 2: site 4:device 5:worker

"id":[1,2] // id is corresponds to the type; type
1:null 2: siteId 4:deviceId 5:workerId

"subDataType":[1], // 1 event 2 Reading(contain
location) 3 online/offline,when type=2, 3 is not
supported

}],

"flag": 1,

"msgId": " GKJoh3WZb873Vx4DBW ",

"msgType": "subscribeRtData", "sign": "\$SIGN",

"sndTime": 1593919901000

}

response {

"data": {}, "errCode": 0,

"flag": 2,

```
"msgId": " GKJoh3WZb873Vx4DBW ",  
"msgType": "subscribeRtData", "sign": "$SIGN",  
"sndTime": 1593919908376
```

```
}
```

7.2.3 Server publish real time data

After subscribe, Safety Suite server will publish real time data to the client automatically.

```
request {  
  
  "data": { //only for definition, the content is in sign  
    field "events": [  
  
      {  
  
        "deviceId": 1, //optional, is null when worker's  
        event  
  
        "eventCode": 11, //see appendix " time":  
        1571030843000,  
  
        "startFlag": 1, //1 start 2 stop "type": 1, //1 alarm  
        2 alert  
  
      }  
  
    ],  
  
    "workerId": 123, "gps":  
  
    {  
  
      "lat": 11,  
  
      "lng": 222  
  
    },  
  
    "time": 1593919910376,  
  
    "online": 1, //1 online 0 offline optional "reading": [  
  
    {  
  
      "data": [  
  
      {
```

```

"decimalPoint": 0,
"detectionMode": 0, "name": "CO",
"unit": "ppm", "val": 0.0
}
],
"online":1, //1 online 0 offline optional "deviceId":
1234,
"time": 1593919910376,
"gps":{"lng":123, "lat":345}
}
]
},
"flag": 1,

```

```

"msgId": " GKJoh3WZb873Vx4DBW ",
"msgType": "subscribeRtData", "sign": "$SIGN",
"sndTime": 1593919913766
}

```

```

response {
"data": {}, // only for definition, the content is in
sign field
"flag": 2,
"msgId": "GKJoh3WZb873Vx4DBW",
"msgType": "subscribeRtData", "sign": "$SIGN",
"sndTime": 1593919916766
}

```

7.3 Get historical data

Sensor attributes definition:

Values Description

val Instrument sensor readings

unit Instrument sensor unit

name Instrument sensor name

decimalPoint reading accuracy, for example:
val=10,

decimalPoint=1 means the value is 1.0

Detection Mode

TubeSampling = 1; Sensor Tube Sampling

WarmingUp = 2; Sensor Warming-up

DisabledTemporary = 3; Sensor disabled
temporary

AlarmOccurring=255; sensor alarm

a. Get historical device data

Getting historical device data will help to get historical device data info to learn sensor and GPS information at some point in history and statistical analysis of the data. Third- party system can also filter the data by site, worker or device info with a period of time and paging information.

POST /third/gas/v1/ Get historical device data over a period of time and filter by

getHisDeviceData site/worker/device

Request content-type= application/json {

"appid": "98bcd21-68c1-4216-84a4-d99d30cb61db", "data": { //only for definition, the content is in sign field

"siteId":1 //option, null for all "workerId":1 //option, null for all "deviceId":3 //option, null for all

"startTime": 1593919901000, // option, historical data time interval start time, using timestamp

"endTime": 1593920501000, // option, historical data time interval end time, using timestamp

```
"pageNo": 1, // option, default 1
"pageSize": 1000, //option, default 1000 ,and
1000 is maximum
},
"random": 2457, //number, int4 "sign": "$SIGN"
}
```

Response content- type= application/text {

```
"rtData": {
"total": 51538, // total size "currentPageSize":
538, //if currentPageSize <
pageSize, that means this is the last page
"currentPageNo": 52,
"deviceData": [
{
"serialNo": "M03A10000067",
"deviceId": 1704,
"time": 1612337283570,
"data": [
{
"detectionMode": 0,
"val": 0.0,
"decimalPoint": 0, "name": "LEL",
"unit": "%LEL"
},
{
"detectionMode": 0,
"val": 0.0,
"decimalPoint": 0, "name": "CO",
"unit": "mg/m3"
},
}
```

```
{  
  "detectionMode": 0,  
  "val": 0.0,  
  "decimalPoint": 1, "name": "H2S",  
  "unit": "mg/m3"  
}
```

```
],
```

```
"gps": {
```

```
  "lat": 31.196919,
```

```
  "lng": 121.609875
```

```
},
```

```
"siteId": null, "workerId": 1258
```

```
}
```

```
]
```

```
}
```

```
}
```

//if response http status code is 400 or 403,
please modify parameters as requested

b. Get historical alarm/alert data

Getting historical alarm/alert data will help to get historical event detail info to learn what event happened at some point in history and statistics analysis of the data. Third-party system can also filter the data by site, worker or device info with a period of time and paging information.

POST /third/gas/v1/ getHisEventData Get historical alarm/alert event data over a period of time and filter by site/worker/device

Request content-type= application/json {

```
"appid": "98bcd21-68c1-4216-84a4-d99d30cb61db", "data": { //only for definition, the
```

content is in sign field "type":1, // option,Search
historical data type:

1- alarm

2- alert null-all

"startTime": 1593919901000, // option, historical
data time interval start time, using timestamp

"endTime": 1593920501000, // Option, historical
data time interval end time, using timestamp

"pageNo": 1, // Required, Page no , from 1

"pageSize": 100, // Required, page size, 100 is
maximum

"siteId": 2, // Option, Site id, Null for all

"workerId": 3, // Option, Worker id, Null for all

"deviceId": 4, // Option, Device id , Null for all

},

"random": 2457, // number, int4 "sign": "\$SIGN"

}

Response content- type= application/text {

"rtData": {

"total": 51538, // total size "currentPageSize":
538, //if currentPageSize <

pageSize, that means this is the last page

"currentPageNo": 52,

"events": [

{

"id": 24008,

"event": "SAS_FAIL", //event type name

"eventCode": 10, //event type id, see appendix

"eventCategory": 1, // 1 SensorAlarmStatus, 2

UserAlarmStatus, 3 WorkerDownPhase, 4

InstrumentFaultStatus

```
"reported": 1587115096952, "serialNo":  
"K0185619080041-RTN",  
"doneBy": "asdasd", "type": 2,//1 alarm 2 alert  
"sensorName": ["mA"], "gps": {  
"lat": "31.92700114569806",  
"lng": "118.861848581693"  
},  
"sensorList": [{  
"idx": 0,  
"alarm": [10],  
"detectionMode": 0,  
"val": 0.0,  
"decimalPoint": 0, "name": "mA",  
"unit": "mg/m3"  
}],  
"equipmentType": "Controller", "modelName":  
"Honeywell Touchpoint Plus", "zoneName": "demo  
area 01",  
  
"timeElapsed": 419,// time of duration(ms)  
"siteName": "nanjing site",  
"workerId": 151,//worker id  
"workerName": "asdasd", "equipmentId": 916,//  
device id "modelId": "2958032896",  
"equipmentTypeId": 2  
}  
]  
}}
```

//if response http status code is 400 or 403,
please modify parameters as requested

7. SAMPLE CODE

You can contact support team to get sample code in Java or Python.

8.1 Java code example

8.1.1 Encrypt/decrypt java code example

```
public static String decrypt(String base64Data,
String base64SecretKey, String alg,String ivStr)
throws Exception {

byte[] secretKey = new
BASE64Decoder().decodeBuffer(base64SecretKey);

if (AES.equals(alg)) {
if (ivStr==null) {
throw new IllegalArgumentException("ivStr is
null");
}

byte[] btlv = ivStr.getBytes(); if (btlv.length<16) {
throw new IllegalArgumentException("ivStr len is
err");
}

if (btlv.length>16) {
byte[] btlv1 = new byte[16];
System.arraycopy(btlv, 0, btlv1, 0, 16); btlv =
btlv1;
}

return decryptAes(base64Data, secretKey,btlv);
}

return null;
}

public static String decryptAes(String
base64Data, /*String secretKey*/byte[]
```

```

enCodeFormat,byte[] ivBt) throws
NoSuchProviderException,
InvalidAlgorithmParameterException {
try {
byte[] encryptedBytes = new
BASE64Decoder().decodeBuffer(base64Data);

//byte[] enCodeFormat = secretKey.getBytes();
SecretKeySpec secretKeySpec = new
SecretKeySpec(enCodeFormat, AES);

IvParameterSpec ivParameterSpec = new
IvParameterSpec(ivBt); Cipher cipher =

Cipher.getInstance("AES/CBC/
PKCS5Padding","BC");

cipher.init(Cipher.DECRYPT_MODE,
secretKeySpec,ivParameterSpec);

byte[] result = cipher.doFinal(encryptedBytes);

return new String(result, "UTF-8");

} catch (IOException |
NoSuchAlgorithmException |
NoSuchPaddingException | InvalidKeyException
| IllegalBlockSizeException |
BadPaddingException e) {

logger.error("", e);

}

return null;

}

```

8.1.2 Get historical data java code example

8.1.3 Websocket connection establish java code

example

8.1.4 Real time instrument readings subscribe

java code example

8.2 Python code example

8.2.1 Encrypt/decrypt python code example

8.2.2 Get historical data java code example

8.2.3 Websocket connection establish python
code example

8.2.4 Real time instrument readings subscribe
python code example

8.3 System Requirements

8.3.1 Hardware Requirements

- CPU: 2-core or more
- RAM: 4GB or more
- Hard disc: 10GB or more
- Internet Access

8.3.2 Software Requirements

- Operating system: No limit

-
- IDE: No limit

Terms and Conditions (<https://www.honeywell.com/terms-conditions>) | Privacy Policy

(<https://www.honeywell.com/privacy-statement>) | © 2023 Honeywell International Inc.

(<http://www.honeywell.com>)

8. APPENDIX
